gen_test_mtls.sh

gen_test_mtls.sh — hermetic mTLS test-cert + pg-password generator

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Script:** tests/observability/gen_test_mtls.sh **Companion of:** tests/observability/metrics_scrape_test.sh → docs/scripts/run_proxy_challenges.md

Overview

`gen_test_mtls.sh` generates — with `openssl`, fully offline — the material the control-API needs to **start** so the conductor can boot the `proxy-api` service (docker-compose.observability.yml) and prove the plaintext Prometheus `/metrics` scrape live.

The control-API is **fail-closed**: it will NOT start (and the plaintext `/metrics` listener therefore never binds) unless `buildTLSConfig` loads all three mTLS materials successfully. This is a FACT of the code, not an assumption:

Fact	Source
`buildTLSConfig` requires **all three** paths or it hard-errors	`control-plane/internal/api/tls.go:26-29`
Server cert+key loaded via `tls.LoadX509KeyPair(TLSCert, TLSKey)`	`control-plane/internal/api/tls.go:31`
Client CA read + parsed into a `x509` pool (must contain ≥1 usable cert)	`control-plane/internal/api/tls.go:36-43`
`ClientAuth = tls.RequireAndVerifyClientCert` (mTLS, no plaintext fallback)	`control-plane/internal/api/tls.go:54`
Env → Config mapping (`CONTROL_API_TLS_CERT/KEY/CLIENT_CA`, `CONTROL_API_METRICS_ADDR`)	`control-plane/internal/api/api.go:26-32`

Fact	Source
`Start` calls `buildTLSConfig` **first**, then `startMetricsListener` — the metrics listener starts **only after** TLS config succeeds	`control-plane/internal/api/server.go:169-178`
The plaintext `/metrics` listener is a **separate** `net/http` server bound to `CONTROL_API_METRICS_ADDR` serving ONLY `/metrics` (no mTLS)	`control-plane/internal/api/server.go:133-158,110-114`

Because `Start` runs `buildTLSConfig` before binding the metrics socket, valid mTLS material is a hard precondition for the scrape — a broken/missing cert = no `/metrics` at all. That is exactly what this generator provisions.

> These are **disposable test materials** for a hermetic self-boot only. They are never operator/production keys and MUST NEVER be committed (§11.4.10 / §11.4.30).

SAN / CN requirements (FACT)

The generator mirrors the production test harness convention (`control-plane/internal/api/harness_test.go:138-143`):

- **Server cert** — CN `helix-control-plane`; `extendedKeyUsage=serverAuth`; SANs `DNS:proxy-control-plane, DNS:localhost, IP:127.0.0.1, IP:::1`. The `proxy-control-plane` DNS name matches the container network alias / committed Prometheus scrape target (`docker-compose.observability.yml:99-101,143-144`).
- **Client cert** — CN `admin@helix` (the audit actor); `extendedKeyUsage=clientAuth`; signed by the same test CA the api uses as its client-CA pool.
- **CA cert** — self-signed `CA:TRUE`; signs both leaves. This CA cert IS the `helixproxy_api_client_ca` secret (the api verifies presented client certs against it).

`buildTLSConfig` (`tls.LoadX509KeyPair` + `AppendCertsFromPEM`) does **not** itself validate the server SAN against a hostname — the SANs matter to any mTLS *client* that dials the control port. The plaintext `/metrics` scrape needs no client cert and no SAN match (it is a separate non-TLS listener).

Prerequisites

- `openssl` (≥1.1; verified on OpenSSL 3.5), `bash`, POSIX coreutils.
- No network, no containers, no root — pure local file generation.

Secret-name mapping (FIXED by the observability overlay)

The four Podman-secret **names** are fixed by `docker-compose.observability.yml:63-71,116-130,137`:

Podman secret	Generated file	Consumed as
`helixproxy_api_cert`	`.mtls/server.crt`	`CONTROL_API_TLS_CERT` → `/run/secrets/helixproxy_api_cert`
`helixproxy_api_key`	`.mtls/server.key`	`CONTROL_API_TLS_KEY` → `/run/secrets/helixproxy_api_key`
`helixproxy_api_client_ca`	`.mtls/ca.crt`	`CONTROL_API_TLS_CLIENT_CA` → `/run/secrets/helixproxy_api_client_ca`
`helixproxy_pg_password`	`.mtls/pg_password.txt`	Postgres DSN assembled at runtime (compose `command:` 137-138)

Usage examples

Generate (idempotent — re-runs reuse existing certs so a booted api is not invalidated; `--force` regenerates):

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 \
    bash tests/observability/gen_test_mtls.sh
```

Re-print only the `podman secret create` commands for already-generated material:

```
bash tests/observability/gen_test_mtls.sh --print-secrets-only
```

Output lands in the **gitignored** `tests/observability/.mtls/` (files `0600`, dir `0700`). The script prints the exact secret-create commands to stdout and NEVER prints key bytes or the pg-password value (§11.4.10).

Conductor boot + scrape sequence (owed to the conductor; §11.4.119 single owner)

This script provisions material only — it boots nothing. The full live sequence:

```
# 1) Generate hermetic test material (writes tests/
        observability/.mtls/).
bash tests/observability/gen_test_mtls.sh

# 2) Create the 4 Podman secrets (rootless, §11.4.161) — exact
        commands are
#    printed by step 1; run them (or `podman secret rm <name>`
        first if they exist):
podman secret create helixproxy_api_cert      tests/
        observability/.mtls/server.crt
podman secret create helixproxy_api_key       tests/
        observability/.mtls/server.key
podman secret create helixproxy_api_client_ca tests/
        observability/.mtls/ca.crt
podman secret create helixproxy_pg_password   tests/
        observability/.mtls/pg_password.txt

# 3) Boot Postgres + Redis + the control-API on top of the base +
        dynamic stacks.
podman-compose -f docker-compose.yml \
               -f docker-compose.dynamic.yml \
               -f docker-compose.observability.yml \
               --profile dynamic up -d proxy-postgres proxy-redis
        proxy-api

# 4) Prove the live /metrics scrape (GREEN guard; asserts real
        exposition content).
HELIX_OBSERVABILITY_STACK=1 \
HELIX_METRICS_URL=http://127.0.0.1:59090/metrics \
    bash tests/observability/metrics_scrape_test.sh
```

Port `59090` is `METRICS_PORT` (`.env.example:217`), published to loopback by `docker-compose.observability.yml:149`. `proxy-postgres` / `proxy-redis` live in the `dynamic` overlay; `proxy-api` is in the `dynamic` profile and `depends_on` both (`docker-compose.observability.yml:102-103,150-157`).

Edge cases

- **Secret already exists** — `podman secret create` fails on a duplicate name; `podman secret rm <name>` first, then re-create (the script prints a reminder).
- **`--print-secrets-only` before generating** — exits non-zero with a hint (no fake success).
- **Postgres password characters** — generated as hex (`openssl rand -hex 24`) so it is URL-safe inside the DSN (`postgres://user:PW@host/db`); no `@`/`/`/`+` to break URL parsing.
- **Missing `nice`/`ionice`** — the self-re-exec degrades gracefully (nice-only, or no cap) rather than failing.

Internal behaviour

1. Self-re-execs once under `nice -n 19 ionice -c 3` (§12.6 host resource cap).
2. `umask 077`, `mkdir -p` + `chmod 700` the output dir (§11.4.10).
3. Generates: test CA (P-256, `CA:TRUE`) → server leaf (serverAuth + SANs) → client leaf (clientAuth) → random pg password (hex).
4. Verifies the material fail-closed: server/client certs parse, and the client cert `openssl verify -CAfile ca.crt` succeeds (mirrors what the api's client-CA pool will do).
5. Prints the 4 `podman secret create <name> <path>` commands (paths only).

Related scripts

- `tests/observability/metrics_scrape_test.sh` — the RED/GREEN live scrape guard this generator enables.
- `docs/scripts/run_proxy_challenges.md` — the proxy Challenge runner runbook.

Sources verified

- `control-plane/internal/api/tls.go`, `api.go`, `server.go` (read 2026-07-01) — cert env vars, fail-closed requirements, metrics-listener ordering.
- `control-plane/internal/api/harness_test.go` (read 2026-07-01) — SAN/CN convention.
- `docker-compose.observability.yml` (read 2026-07-01) — secret names, ports, service dependencies.

Last verified date: 2026-07-01